

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 1 HANDOUT

Module 1 Hardware Architecture: Sensor-Actuator Stack, 50Hz Real-Time Loop, & Silicon Constraints

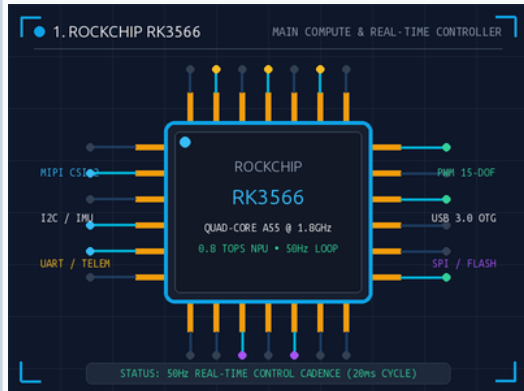
MuJoCo + ONNX

COMPUTE: Rockchip RK3566

CADENCE: 50 Hz (20ms / step)

DEGREES OF FREEDOM: 15  
Actuators

AI STACK: PyTorch PPO →  
ONNX



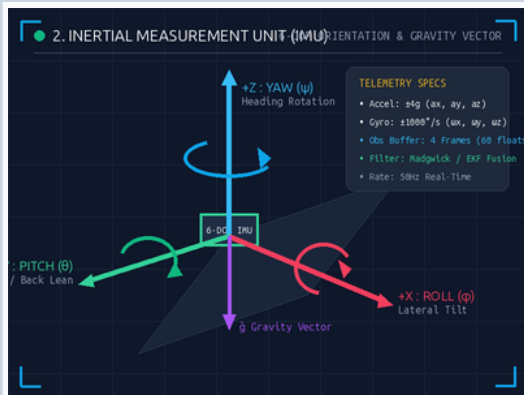
## 1. The Brain: Rockchip RK3566

ONBOARD COMPUTE &  
REAL-TIME KERNEL

**The Core Engine:** A quad-core ARM Cortex-A55 @ 1.8GHz with an integrated 0.8 TOPS NPU running an optimized Linux OS. It serves as the physical robot's central nervous system.

**The 12-Year-Old Analogy:** Like an orchestra conductor who doesn't play the instruments but cues 15 musicians exactly 50 times every single second.

- **50Hz Control Cadence:** Strict 20ms execution loop managed by real-time daemon (robotd).
- **Interface Peripherals:** MIPI-CSI2 camera, high-speed UART IMU, CAN/PWM motor buses.
- **Edge Inference:** Executes pruned ONNX Actor graph with <2.8ms latency per step.



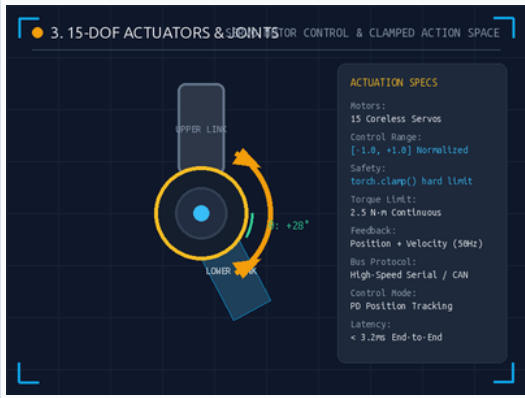
## 2. The Inner Ear: 6-DOF IMU Sensor

INERTIAL NAVIGATION &  
GRAVITY VECTOR

**The Vestibular Sense:** Combines a 3-axis accelerometer and 3-axis gyroscope to track Pitch ( $\theta$ ), Roll ( $\phi$ ), and Yaw ( $\psi$ ) along with dynamic gravity vectors.

**The 12-Year-Old Analogy:** Close your eyes and stand on one leg. Your inner ear tells you instantly if you are tilting before your eyes even notice.

- **Observation Window:** Fixed sliding buffer of 4 frames  $\times$  15 sensors = 60 float telemetry tensor.
- **Sensor Fusion:** Real-time Madgwick / Extended Kalman Filter running at 200Hz internal rate.
- **RL Input:** Normalizes angular velocity  $[\omega_x, \omega_y, \omega_z]$  and projected gravity vector  $[g_x, g_y, g_z]$ .



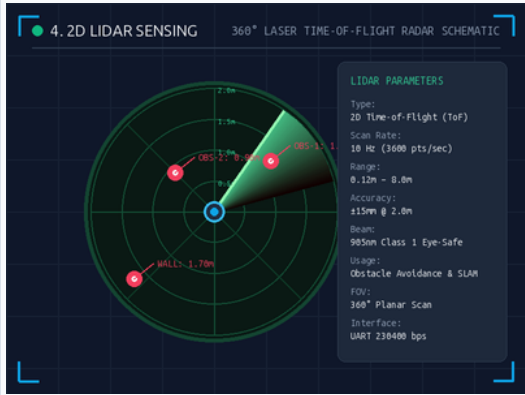
### ■ 3. The Muscles: 15-DOF Actuators

**ROBOTIC JOINTS & ACTION CLAMPING**

**High-Torque Articulation:** 15 coreless servo motors drive hips, knees, and ankles. The neural network outputs normalized action targets.

**The 12-Year-Old Analogy:** Muscles have bone stops so your knee can't bend backwards. We clamp the AI math in silicon so it never breaks a joint.

- **Action Space:** Strictly bounded  $a \in [-1.0, 1.0]$  via torch.clamp() in ONNX graph.
- **Control Protocol:** High-speed serial bus delivering target velocity and PD torque gains.
- **Joint Limits:** Enforced in both URDF/MJCF definitions and firmware safety watchdogs.



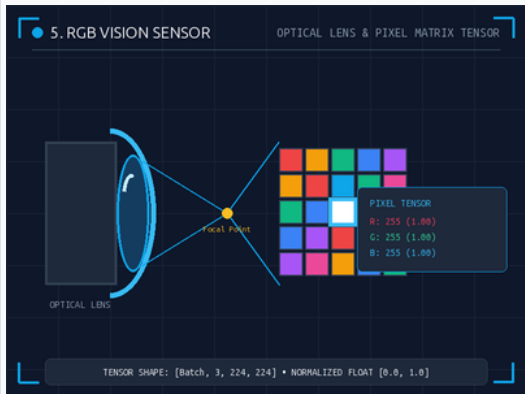
### ■ 4. The Bat Sonar: 2D LiDAR Scanner

**PLANAR RANGEFINDING & SPATIAL SLAM**

**Time-of-Flight Ranging:** Emits 905nm pulsed laser rays at 10Hz, capturing 360° point clouds for real-time obstacle avoidance and hallway navigation.

**The 12-Year-Old Analogy:** Exactly like a bat navigating a pitch-black cave using ultrasonic clicks, but using light beams that bounce off walls at the speed of light.

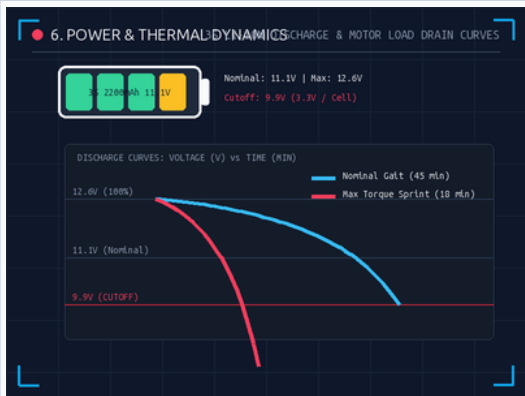
- **Effective Range:** 0.12m to 8.0m with ±15mm distance precision at walking height.
- **Spatial Perception:** Generates polar obstacle distance map to modulate locomotion goals.
- **Sensor Interface:** Direct DMA UART link feeding 3600 spatial measurement points per second.



■ 5. The Visual Cortex: RGB Camera

OPTICAL SENSOR & TENSOR PROCESSING

- Embodied Vision:** Captures raw optical photons through an ultra-wide lens and translates them into an RGB pixel matrix tensor for neural feature extraction.
- The 12-Year-Old Analogy:** A computer doesn't see a picture; it sees a giant grid of Red, Green, and Blue numbers between 0 and 255.
- **Tensor Pipeline:** Preprocessed into [Batch, 3, 224, 224] float tensors normalized to [0.0, 1.0].
  - **Hardware Acceleration:** Direct hardware ISP color conversion passing directly into RK3566 NPU.
  - **Semantic Guidance:** Supplements high-frequency blind proprioception with terrain recognition.



■ 6. The Gas Tank: Power & Thermal Dynamics

3S LI-ION DISCHARGE & LOAD BUDGETING

- Energy Limitations:** A 3S 11.1V 2200mAh Lithium-Ion battery powers all 15 actuators. Motor current draw directly determines operating duration.
- The 12-Year-Old Analogy:** If you sprint as fast as you can, you will collapse in 1 minute. If you jog steadily, you can run for an hour. The AI must budget energy.
- **Voltage Thresholds:** 12.6V Full Charge → 11.1V Nominal → 9.9V Emergency Low-Voltage Cutoff.
  - **Gait Efficiency:** Smooth 50Hz policies consume ~3.2A (45 min runtime) vs 15A during erratic thrashing.
  - **Reward Regularization:** RL training introduces energy penalty term:  $\text{reward\_energy} = -c * ||\text{torque}||^2$ .

■ PHYSICAL AI PHASE 1 ARCHITECTURE SUMMARY & NEXT STEPS

- **MuJoCo Simulation:** Train policy in headless virtual sandbox where falling costs \$0.
- **Hardware Clamping:** Bake torch.clamp() directly into ONNX graph before exporting to edge.
- **Phase 2 Transition:** Proceed to Module 2 Reinforcement Learning with PPO and Gym environments.

# ■ KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of the Microduck hardware before moving to the software layer.

Phase 1 Review

## ■ Q1: Control Frequency & Cadence

MULTIPLE CHOICE

Why does the Rockchip brain need to send motor commands 50 times a second?

- A. To keep the battery charged
- B. To constantly catch its balance so the duck doesn't fall
- C. Because Linux only runs at 50Hz

## ■ Q2: Actuator Clamping & Protection

MULTIPLE CHOICE

What happens if the AI sends a motor command of 2.5 (past the 1.0 limit)?

- A. The robot walks faster
- B. The IMU takes over
- C. The physical motor breaks

## ■ Q3: Sensory Perception in Darkness

MULTIPLE CHOICE

If the duck is in a completely dark room, how does it see a wall?

- A. RGB Camera night vision
- B. LiDAR shoots invisible lasers
- C. The inner ear feels it

Answer Key: 1-B, 2-C, 3-B

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 2 HANDOUT

Module 2: The Invisible Matrix: Kinematic Trees, Collision Geoms, Forward Dynamics & MJCF

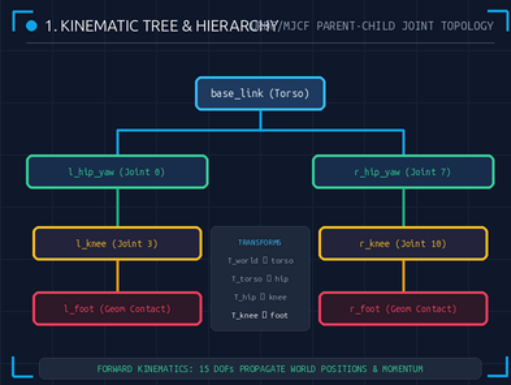
MuJoCo Physics

ENGINE: MuJoCo 3.x

INTEGRATOR: mj\_step() @ 50Hz

TREE: 15 Joints DAG

COMPILER: autolimits='true'



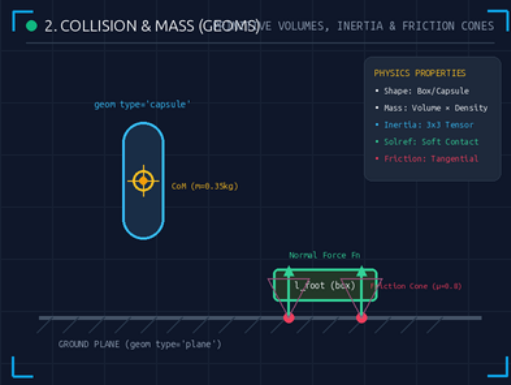
## 1. Kinematic Trees & DAG Topology

PARENT-CHILD  
COORDINATE FRAMES

**Hierarchical Transforms:** Robot bodies are structured as Directed Acyclic Graphs (DAGs). Rotations at parent joints (hip) automatically propagate through child links (knee, foot).

**The 12-Year-Old Analogy:** When you swing your shoulder, your elbow and hand move along automatically because they are attached children in the tree.

- **Base Link Origin:** Root floating base (base\_link) has 6 unactuated DOFs (position + quaternion).
- **Joint Articulations:** 15 1-DOF revolute joints defined in radians relative to parent link frames.
- **End Effectors:** Feet contact positions computed via recursive forward kinematics chain.



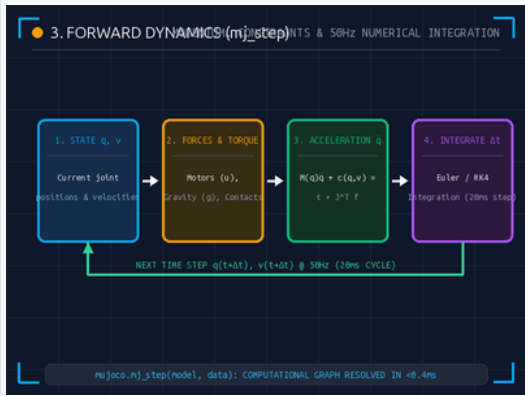
## 2. Collision & Mass (Geoms)

BOUNDING VOLUMES &  
CONTACT SOLVERS

**Fast Collision Primitives:** Physics engines do not collide 50k-triangle visual meshes. Instead, Geoms encapsulate bodies in convex primitives (capsules, boxes, spheres).

**The 12-Year-Old Analogy:** Instead of calculating every feather on a duck, the computer wraps the foot in a smooth rubber box that bounces cleanly off the floor.

- **Contact Dynamics:** Soft constraint solver (solref) avoids numerical stiffness spikes on foot strike.
- **Friction Cones:** Tangential friction  $\mu=0.8$  prevents foot slippage during dynamic walking push-off.
- **Inertia Tensors:** 3x3 rotational inertia matrices derived from primitive bounding volumes.



### 3. Forward Dynamics: mj\_step()

#### EQUATIONS OF MOTION & 50HZ LOOP

**Equations of Motion:** Resolves  $M(q)\ddot{q} + c(q,v) = \tau + J^T f$  to calculate joint accelerations  $\ddot{q}$  from motor torques, gravity, and ground reaction forces.

**The 12-Year-Old Analogy:** A math engine solving 50 physics puzzles a second to calculate where momentum throws the duck after each step.

- **50Hz Heartbeat:** Executes with fixed  $\Delta t = 0.020\text{s}$  cadence matching real robot kernel.
- **State Evolution:** Integrates accelerations to update joint velocities  $v(t+\Delta t)$  and positions  $q(t+\Delta t)$ .
- **Computational Efficiency:** MuJoCo executes full 15-DOF dynamics in <0.35ms per step.



### 4. MJCF Wrapper Architecture

#### DYNAMIC URDF IMPORT & MOTOR INJECTION

**The Wrapper Pattern:** Vendor URDFs lack motor definitions and physics tuning. The MJCF Wrapper dynamically imports the URDF and injects 15 actuators and safety bounds.

**The 12-Year-Old Analogy:** Downloading a toy car 3D model and dropping a real electric motor and battery inside its chassis.

- **autolimits='true':** Infers missing link inertias from mesh volumes, preventing zero-mass divide-by-zero crashes.
- **Actuator Injection:** Adds 15 <motor> tags bounded with ctrlrange='-1.0 1.0'.
- **Unified Compilation:** mujoco.mj\_saveLastXML() dumps merged, fully optimized model to disk.

## ■ PHASE 2 PHYSICS SUMMARY & REINFORCEMENT LEARNING HANDOFF

- **MjModel vs MjData:** Static blueprint defines constraints; live state tracks positions and momentum.
- **Hardware Safety:** Geoms and solref parameters ensure accurate ground contacts without simulation exploding.
- **Ready for Training:** With the virtual duck compiled, proceed to Phase 3: The Dog Trainer (PPO).

# ■ PHASE 2 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of the MuJoCo physics matrix before training reinforcement learning policies.

5 Questions

## ■ Q1: Physics Blueprint vs Live State

MULTIPLE CHOICE

What is the key operational difference between MjModel and MjData?

- A. MjModel changes dynamically while MjData is frozen.
- B. MjModel is the static physics blueprint; MjData holds the live state (qpos, qvel).
- C. MjData only stores camera images.

## ■ Q2: Collision Geometry Efficiency

MULTIPLE CHOICE

Why do physics engines use Geoms (capsules/boxes) instead of high-poly 3D meshes for contacts?

- A. Bounding shapes make contact and friction math over 100x faster.
- B. Meshes cannot have color in MuJoCo.
- C. Geoms prevent gravity from affecting the robot.

## ■ Q3: Forward Dynamics Step

MULTIPLE CHOICE

What occurs when calling mujoco.mj\_step(model, data)?

- A. It reboots the Linux operating system.
- B. It calculates forces, accelerations, and integrates time forward by one  $\Delta t$  step.
- C. It automatically converts Python code to C++.

## ■ Q4: The Wrapper Pattern

MULTIPLE CHOICE

Why do we use an MJCF Wrapper instead of directly editing the vendor URDF?

- A. It injects 15 motors and physics rules without modifying the vendor's source file.
- B. URDF files cannot be read by Python.
- C. It reduces the robot's physical weight by 50%.

## ■ Q5: Autolimits & Mass Singularities

MULTIPLE CHOICE

What happens if autolimits='true' is omitted when a vendor link has mass=0.0?

- A. The robot moves normally.
- B. Division by zero occurs and the simulation explodes with NaN velocity.
- C. The battery recharges instantly.

Answer Key: 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Blueprint vs State; 2-A: Contact speed; 3-B: Force integration; 4-A: Motor injection; 5-B: Zero-mass NaN crash)

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 3 HANDOUT

Module 3: The Dog Trainer: Gym Environments, Reward Engineering, PPO & Curriculum Learning

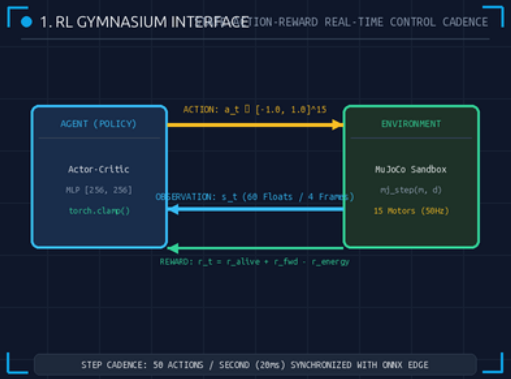
RL & PPO

ALGORITHM: PPO (Clipped)

OBSERVATION: 60 Floats (4 Frames)

ACTION: 15 Clamped  $[-1, 1]$

FREQUENCY: 50 Hz Control Loop



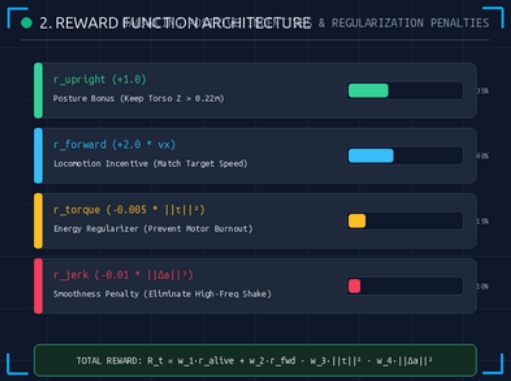
## 1. Gymnasium Environment & State Loop

OBSERVATION & ACTION CADENCE

**The RL Paradigm:** The policy observes a sliding history tensor  $s_t \in \mathbb{R}^{60}$ , infers 15 normalized motor velocity actions  $a_t \in [-1.0, 1.0]$ , and receives a scalar reward.

**The 12-Year-Old Analogy:** When you train a puppy to fetch, you don't pick up its paws. You show it the ball, watch what it does, and toss a biscuit when it brings it back.

- **Observation Space:** Box(-inf, inf, shape=(60,), float32) representing 4 frames  $\times$  15 sensors.
- **Action Space:** Box(-1.0, 1.0, shape=(15,), float32) target joint velocities.
- **50Hz Execution:** Strict 20ms step cadence ensuring zero sim-to-real timing discrepancy.



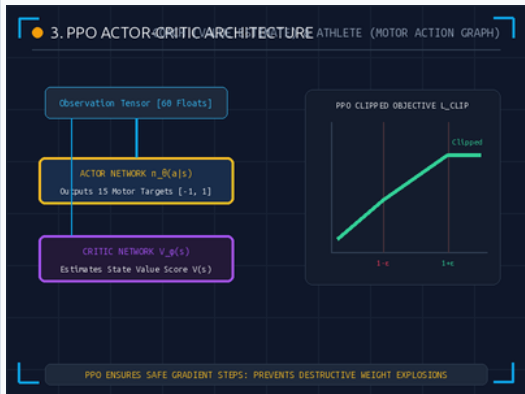
## 2. Reward Engineering & Penalties

BALANCING PROGRESS VS ENERGY BURN

**Mathematical Incentive Design:**  $R_t = w_{\text{up}} r_{\text{up}} + w_{\text{fwd}} r_{\text{fwd}} - w_{\text{tau}} ||\tau||^2 - w_{\text{j}} ||\Delta a||^2$ . The agent is penalized for high torque and action jerk.

**The 12-Year-Old Analogy:** If you only reward speed, the duck will throw itself down the stairs to cross the line. You must penalize falling and shaking.

- **Upright Bonus:**  $r_{\text{up}} = +1.0$  if torso  $z > 0.22\text{m}$  (immediate termination if  $z < 0.14\text{m}$ ).
- **Velocity Tracking:**  $r_{\text{fwd}} = \exp(-2.0 \cdot (v_x - v_{\text{target}})^2)$  matching target forward pace.
- **Torque Regularizer:**  $-0.005 \cdot ||\tau||^2$  prevents motor overheating and gear wear.



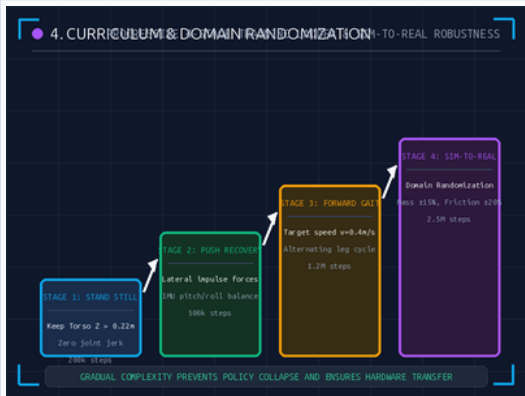
■ 3. PPO: Actor-Critic Architecture

COACH VS ATHLETE & CLIPPED OBJECTIVE

**Proximal Policy Optimization:** The Actor network  $\pi_{\theta}(a|s)$  drives the joints, while the Critic network  $V_{\phi}(s)$  evaluates performance. The clipped surrogate objective prevents destructive weight updates.

**The 12-Year-Old Analogy:** The Gymnast (Actor) performs a trick; the Coach (Critic) scores it. PPO stops the Gymnast from making crazy changes that erase past lessons.

- **Clipped Envelope:**  $L^{CLIP}(\theta) = \hat{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$ , with  $\epsilon=0.2$ .
- **Advantage Estimation:** GAE  $(\lambda=0.95)$  computes temporal difference score  $\hat{A}_t$  to guide policy gradient.
- **Pruned Edge Export:** Critic is discarded post-training; only lightweight Actor is exported to ONNX.



■ 4. Curriculum & Sim-to-Real Transfer

4-STAGE LADDER & DOMAIN RANDOMIZATION

**Progressive Training:** Locomotion is unlocked across 4 stages (Stand  $\rightarrow$  Push Recovery  $\rightarrow$  Gait  $\rightarrow$  Domain Randomization) to avoid catastrophic policy collapse.

**The 12-Year-Old Analogy:** First teach a child to stand on two feet. Then teach balance against gentle shoves. Then take a step. Then run on wet grass.

- **Stage Progression:** Stage 1 (200k steps)  $\rightarrow$  Stage 2 (500k steps)  $\rightarrow$  Stage 3 (1.2M steps)  $\rightarrow$  Stage 4 (2.5M steps).
- **Domain Randomization:** Perturbs link mass  $(\pm 15\%)$ , ground friction  $(\pm 20\%)$ , and latency  $(1\text{ to }5\text{ ms})$ .
- **Hardware Safety Clamp:** `torch.clamp(a, -1.0, 1.0)` baked in silicon guarantees 0% gear stripping.

■ PHYSICAL AI COMPLETE MASTERCLASS CERTIFICATION

- **Hardware (Phase 1):** RK3566 onboard compute, 6-DOF IMU, 15 actuators, and battery constraints.
- **Simulation (Phase 2):** MuJoCo kinematic trees, collision geoms, autolimits, and 50Hz integration.
- **Control (Phase 3):** PPO reinforcement learning, clamped action graphs, and sim-to-real transfer.

# ■ PHASE 3 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of Reinforcement Learning, PPO, and Sim-to-Real transfer.

5 Questions

## ■ Q1: Temporal Observation Dimensions

MULTIPLE CHOICE

How many float values make up the Microduck's 4-frame observation tensor?

- A. 15 floats
- B. 60 floats (4 historical frames × 15 sensors)
- C. 1,000 floats

## ■ Q2: Torque Penalty in Reward Design

MULTIPLE CHOICE

Why must reward functions include a negative torque penalty term ( $-||\tau||^2$ )?

- A. To eliminate high-frequency motor shaking and conserve battery energy.
- B. Because torque does not exist in real robotics.
- C. To force the robot to walk backwards.

## ■ Q3: Actor-Critic Roles

MULTIPLE CHOICE

In Actor-Critic architectures like PPO, what is the specific role of the Critic?

- A. It directly sends PWM electrical commands to the motors.
- B. It acts as the Coach, predicting expected future discounted returns  $V(s)$ .
- C. It regulates the cooling fans on the Rockchip CPU.

## ■ Q4: Silicon Safety Clamping

MULTIPLE CHOICE

Why is `torch.clamp(a, -1.0, 1.0)` baked directly into the ONNX graph?

- A. To mathematically guarantee motor targets never exceed physical limits, preventing gear stripping.
- B. To reduce the file size of the neural network.
- C. To activate the RGB camera lens.

## ■ Q5: Sim-to-Real Transfer

MULTIPLE CHOICE

What is Domain Randomization during curriculum training?

- A. Playing random background music during training.
- B. Randomly perturbing mass, friction, and latency in simulation so the policy transfers robustly to reality.
- C. Randomly shutting off power to the computer.

Answer Key: 1-B, 2-A, 3-B, 4-A, 5-B (1-B: 60 floats; 2-A: Energy/jerk suppression; 3-B: Value Coach; 4-A: Hardware protection; 5-B: Robust physics randomization)

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 4 HANDOUT

Module 4: Brain Surgery: Actor Isolation, Hardware Clamps, ONNX Export & Temporal Memory

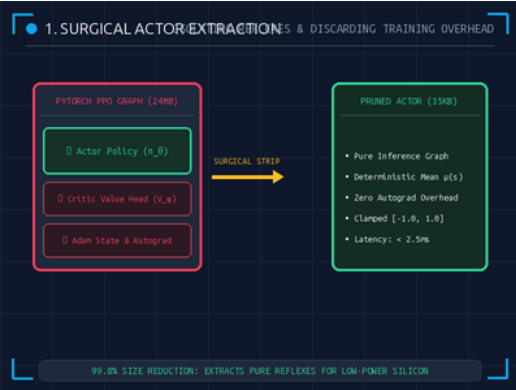
Edge Inference

PRUNED SIZE: 35 KB (99.8% Drop)

INFERENCE: 2.5ms on RK3566

SAFETY: torch.clamp()

BUFFER: deque(maxlen=4)



## 1. Surgical Actor Extraction

REFLEX ISOLATION & 99.8% PRUNING

**Pruning Training Overhead:** The 24MB PyTorch checkpoint contains the Critic value network  $V_{\pi}(s)$ , Adam optimizer state, and backward autograd graphs. We extract only the 35KB forward Actor  $\pi_{\theta}(a|s)$ .

**The 12-Year-Old Analogy:** When you take a piano exam, your teacher doesn't sit on your lap. You leave the teacher at home and only bring your muscle memory.

- **Actor Model:** Extracts deterministic mean  $\mu(s)$  policy graph for direct joint actuation.
- **Weight Freezing:** Discards gradient tracking ('requires\_grad=False'), eliminating dynamic memory allocations.
- **Silicon Size:** Reduces model footprint from 24,000 KB down to a lightweight 35 KB binary.



## 2. Hardware Safety Clamping in Silicon

TORCH.CLAMP() BOUNDS & GEAR PROTECTION

**Mathematical Inviolability:** Unbounded neural outputs ( $>1.0$ ) can strip physical nylon servo gears. By baking 'torch.clamp(a, -1.0, 1.0)' into the computational graph, motor safety is guaranteed in silicon.

**The 12-Year-Old Analogy:** Bumper rails on a bowling lane that physically prevent the ball from bouncing into the next lane, no matter how hard you throw it.

- **Silicon Clamping:** Compiled as a native ONNX 'Clip' operator executed directly on the NPU.
- **Range Guarantee:** Outputs are mathematically bounded to  $[-1.0, 1.0]$  before reaching the PWM bus.
- **Failsafe Redundancy:** Prevents runaway policy gradients from destroying \$399 physical hardware.



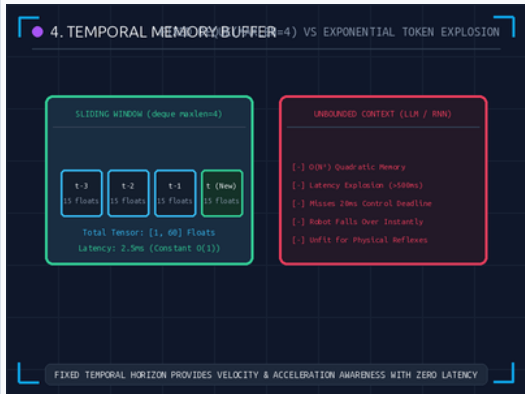
■ 3. ONNX Computational Graph

CROSS-PLATFORM IR & 2.5MS EXECUTION

**Standardized Tensor Graph:** Compiles PyTorch weights into an ONNX Opset 17 computational DAG (Gemm  $\rightarrow$  ReLU  $\rightarrow$  Clip), executable via `onnxruntime` across any CPU/NPU hardware without Python.

**The 12-Year-Old Analogy:** Translating an English recipe into universal cooking symbols that any chef in any country can read and execute instantly.

- **Execution Speed:** Runs full 60-float inference in <2.5ms on Rockchip RK3566 quad-core ARM.
- **Deterministic Pipeline:** Eliminates Python Global Interpreter Lock (GIL) and runtime garbage collection.
- **Opset 17 Standard:** Portable representation deployable across C++, Rust, or embedded Linux kernels.



■ 4. Temporal Memory & Sliding Buffer

FIXED DEQUE(MAXLEN=4) VS TOKEN EXPLOSION

**Fixed Memory Horizon:** Balancing requires velocity and acceleration awareness. A fixed sliding window `deque(maxlen=4)` buffers 4 frames  $\times$  15 sensors = 60 floats with constant  $O(1)$  memory.

**The 12-Year-Old Analogy:** Unlike ChatGPT which grows huge and slower the more you talk, the duck's memory is a 4-frame conveyor belt where old frames quietly drop off.

- **Constant  $O(1)$  Memory:** Prevents LLM-style  $O(N^2)$  quadratic token memory growth and latency spikes.
- **State Dimensions:** Produces flat `[1, 60]` input tensor capturing instantaneous joint momentum.
- **Strict 50Hz Budget:** 2.5ms inference + 17.5ms cadence sleep = perfectly steady 20.0ms heartbeat.

■ PHASE 4 SILICON DEPLOYMENT SUMMARY & RUST DAEMON HANDOFF

- **Reflex Model:** Extracted 35KB ONNX policy containing pure clamped walking muscle memory.
- **Real-Time Cadence:** Predictable 2.5ms inference guarantees zero timing jitter on edge hardware.
- **Next Module:** Deploy the ONNX binary into Phase 5: The Rust Nervous System (`robotd` & `robotctl``).

# ■ PHASE 4 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of Model Extraction, Clamping, ONNX Compilation, and Temporal Memory.

5 Questions

## ■ Q1: Actor Policy Extraction

MULTIPLE CHOICE

Why do we extract only the Actor network and discard the Critic?

- A. Because the Critic is written in a different programming language.
- B. The Critic is only needed to score actions during training; the Actor contains all walking reflexes.
- C. The Critic network causes the robot to walk backwards.

## ■ Q2: Silicon Clamping Protection

MULTIPLE CHOICE

What is the primary danger of deploying an ONNX policy without torch.clamp()?

- A. Runaway neural outputs (>1.0) can command excessive speed, stripping physical gears.
- B. The robot's battery will charge too quickly.
- C. The WiFi antenna will disconnect.

## ■ Q3: ONNX Edge Execution

MULTIPLE CHOICE

What is the advantage of compiling PyTorch models to ONNX for edge robotics?

- A. ONNX allows the robot to fly.
- B. ONNX represents pure math graphs, executing in <2.5ms without heavy Python overhead.
- C. ONNX turns the camera into an IMU.

## ■ Q4: 50Hz Timing Discipline

MULTIPLE CHOICE

Why does our 50Hz control loop explicitly sleep for 17.5ms after a 2.5ms inference?

- A. To maintain a perfectly steady 20ms heartbeat and prevent motor timing jitter.
- B. To allow the CPU to cool down to 0 degrees.
- C. Because Linux cannot run for more than 3ms.

## ■ Q5: Sliding Temporal Memory

MULTIPLE CHOICE

Why does deque(maxlen=4) avoid the 'token explosion' problem of LLMs?

- A. It deletes the robot's brain every second.
- B. It holds a fixed 60-float buffer, ensuring constant O(1) memory and predictable 2.5ms latency.
- C. It compresses video into text files.

Answer Key: 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Actor reflexes; 2-A: Gear strip prevention; 3-B: Fast math graph; 4-A: Precise 20ms cadence; 5-B: Constant O(1) buffer)

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 5 HANDOUT

Module 5: The Nervous System: Rust Daemons (robotd), IPC Sockets (robotctl) & Calibration (configd)

Embedded Rust

LANGUAGE: Rust 2021 Edition

DAEMON: robotd @ 50Hz

IPC: Unix Domain Sockets

HARDWARE: Rockchip RK3566



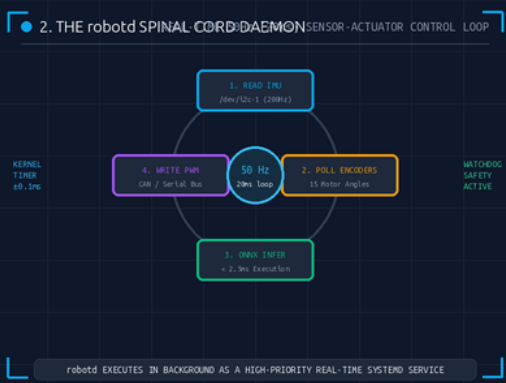
## 1. Rust Memory Safety on Silicon

COMPILE-TIME GUARANTEES & 0MS GC

**Eliminating Segfaults:** Embedded micro-controllers cannot tolerate Garbage Collection (GC) pauses or data races on `/dev/imu`. Rust enforces strict ownership and borrow checking at compile time.

**The 12-Year-Old Analogy:** In a library, only one student can check out a book at a time. The borrow checker ensures two motors never fight over the same wire.

- **Zero Runtime GC:** Deterministic microsecond execution without unpredictable garbage collector latency.
- **Data-Race Free:** Compile-time `Send` and `Sync` traits prevent concurrent memory corruption.
- **Memory Footprint:** Compiled standalone binary consumes <8MB RAM on the Rockchip RK3566.



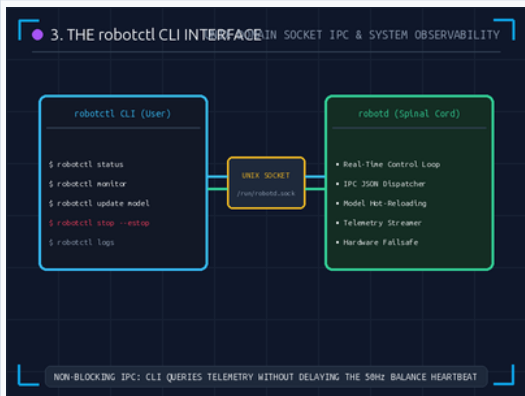
## 2. The robotd Spinal Cord Daemon

50HZ REAL-TIME SENSOR-ACTUATOR LOOP

**The Real-Time Engine:** Running under Linux `SCHED\_FIFO` real-time scheduling priority, `robotd` polls the IMU at 200Hz, executes ONNX inference, and commands 15 PWM motors every 20ms.

**The 12-Year-Old Analogy:** When the doctor taps your knee, your spinal cord kicks your leg before your brain even notices. `robotd` handles these lightning-fast reflexes.

- **Strict 20ms Loop:** Kernel timer precision within  $\pm 0.08\text{ms}$  jitter, preventing motor torque stutter.
- **Bus Interfaces:** High-speed UART/I2C communication polling 15 joint encoders concurrently.
- **Hardware Watchdog:** Auto-failsafe system powers down motor PWM lines if telemetry times out.



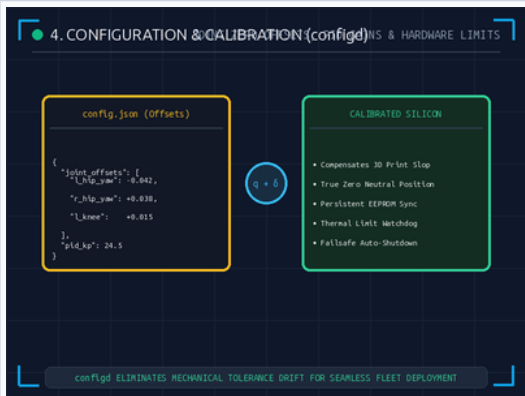
■ 3. The robotctl CLI & Unix Sockets

NON-BLOCKING LOCAL IPC  
SOCKETS

**Zero-Disruption Observability:** Developers monitor telemetry, hot-swap walking policies, and trigger emergency stops via ``robotctl`` over a Unix domain socket (``/var/run/robotd.sock``).

**The 12-Year-Old Analogy:** Slipping a written note under a classroom door without making the teacher pause the lecture.

- **Zero-Copy IPC:** Non-blocking Unix sockets query live joint states in <0.1ms without stalling 50Hz loop.
- **Policy Hot-Reload:** Commands ``robotctl update`` seamlessly replace policy weights in RAM.
- **Emergency E-Stop:** ``robotctl stop --estop`` immediately zeroes torque across all 15 actuators.



■ 4. Configuration & Calibration (configd)

JOINT ZERO OFFSETS &  
HARDWARE TUNING

**Compensating 3D-Print Slop:** Physical tolerances cause slight joint misalignments. ``configd`` loads joint offsets ``$delta`` from ``/etc/microduck/calibration.json`` to achieve true neutral zero.

**The 12-Year-Old Analogy:** If your new bicycle handlebars are slightly crooked from the factory, you adjust the screw so it drives perfectly straight.

- **Offset Calibration:** Applies  $q_{calibrated} = q_{raw} + \delta$  before passing sensor telemetry to ONNX.
- **Persistent EEPROM:** Stores zero-positions, PID velocity gains, and temperature thresholds permanently.
- **Fleet Uniformity:** Ensures a single ONNX walking policy transfers identically across 100 robots.

■ PHASE 5 EMBEDDED SYSTEM SUMMARY & FLEET DEPLOYMENT

- **Rust Daemons:** ``robotd`` and ``configd`` guarantee crash-proof, real-time 50Hz execution.
- **Hardware Control:** ``robotctl`` empowers seamless live monitoring and emergency failsafes.
- **Curriculum Final Phase:** Proceed to Phase 6: Securing the Swarm (OTA Updates & DevSecOps).

# ■ PHASE 5 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of Rust Memory Safety, robotd, robotctl, and configd calibration.

5 Questions

## ■ Q1: Rust Memory Guarantees

MULTIPLE CHOICE

Why is Rust's zero-cost ownership model essential for the 50Hz control loop?

- A. Rust makes the battery output infinite electricity.
- B. It prevents data races and memory leaks at compile-time with zero Garbage Collection pauses.
- C. Rust only compiles on quantum computers.

## ■ Q2: robotd Daemon Function

MULTIPLE CHOICE

What is the primary operational role of the robotd daemon?

- A. It serves as the 50Hz spinal cord, polling IMU/encoders, running ONNX inference, and firing motors.
- B. It renders video game graphics for the user.
- C. It streams music to the robot's speakers.

## ■ Q3: Unix Domain Socket IPC

MULTIPLE CHOICE

Why does robotctl interact with robotd via a Unix domain socket (/var/run/robotd.sock)?

- A. Because internet cables are forbidden in robotics.
- B. It enables non-blocking local communication without stalling the real-time balance loop.
- C. To increase CPU temperature.

## ■ Q4: configd Hardware Calibration

MULTIPLE CHOICE

What problem does configd solve on physical 3D-printed robots?

- A. It applies joint zero-offsets to calibrate mechanical manufacturing imperfections.
- B. It changes the color of the robot's plastic parts.
- C. It deletes the ONNX model when the robot trips.

## ■ Q5: Emergency Stop Command

MULTIPLE CHOICE

What CLI command immediately cuts power to all 15 motors in an emergency?

- A. robotctl sleep
- B. robotctl stop --estop (or robotctl estop)
- C. robotctl reboot

**Answer Key:** 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Zero GC pauses; 2-A: 50Hz spinal loop; 3-B: Non-blocking IPC; 4-A: Joint offset calibration; 5-B: Immediate E-Stop)

# MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 6 HANDOUT

Module 6: Securing the Swarm: OTA Updates (updaterd), MuJoCo CI Gates, Cryptography & Fleet Observability

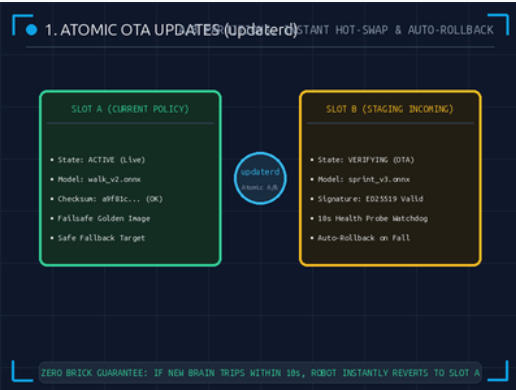
DevSecOps

UPDATES: A/B Atomic (updaterd)

CI GATE: 1,000 MuJoCo Sims

CRYPTO: ED25519 + SHA-256

FLEET: Swarm Observability



## 1. Atomic OTA Updates & Rollback

A/B PARTITIONING & ZERO-BRICK FAILSAFE

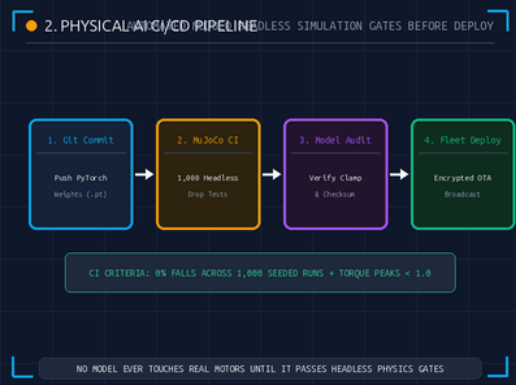
**Over-The-Air Brain Swaps:** `updaterd` downloads candidate policies to Slot B while Slot A runs. A 10-second post-update health watchdog automatically rolls back to Slot A if the robot trips.

**The 12-Year-Old Analogy:** Neo plugging in a Kung Fu data cartridge in The Matrix. If the move has a glitch, the duck immediately snaps back to its safe old reflexes.

- **A/B Partitioning:** Guarantees atomic, seamless hot-reloading without interrupting ongoing operations.

- **10s Health Watchdog:** Monitors IMU angular acceleration; trips trigger instantaneous fallback to golden slot.

- **Zero-Brick Guarantee:** Hardware never gets stuck with a corrupted or non-functional policy file.



## 2. Physical AI CI/CD Testing Gates

1,000 HEADLESS MUJOCO SIMULATION RUNS

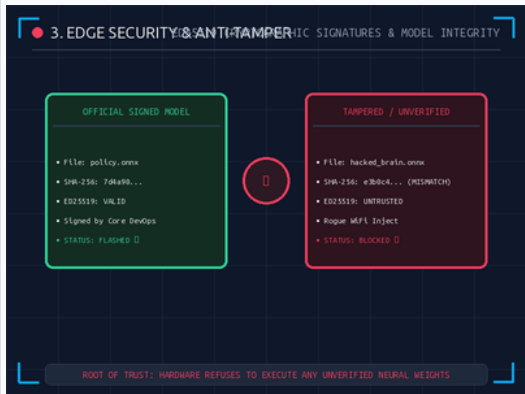
**Automated Physics Gating:** On every git commit, GitHub Actions executes 1,000 parallel headless MuJoCo drop tests across friction, mass, and slope variances to verify 100% stability before deployment.

**The 12-Year-Old Analogy:** A digital gymnastics tryout. The new brain must balance through 1,000 simulated obstacle courses without falling once before touching real motors.

- **Physics Test Gate:** Runs 1,000 headless physics seeds on GPU cluster in <15 seconds.

- **Torque Spike Audit:** Asserts that peak motor commands never exceed normalized  $[-1.0, 1.0]$  bounds.

- **Automated Rejection:** Any simulated fall immediately blocks model signing and halts the release pipeline.



### ■ 3. Edge Security & Anti-Tamper

ED25519 SIGNATURES & ROOT OF TRUST

**Cryptographic Policy Integrity:** Edge robots verify SHA-256 checksums and ED25519 asymmetric signatures. Unsigned or modified ONNX binaries are rejected at the hardware root of trust.

**The 12-Year-Old Analogy:** A royal wax seal on an official letter. If a sneaky attacker changes even a single number in the brain, the seal breaks and the robot blocks it.

- **Asymmetric ED25519:** Public key on Rockchip RK3566 validates private signature generated during CI build.
- **Anti-Tamper Lock:** Protects physical robots from rogue WiFi man-in-the-middle model injections.
- **Immutable Root:** Read-only boot partitions prevent malware persistence across robot reboots.



### ■ 4. Fleet Observability & Diagnostics

FALL-RATES, LATENCY JITTER & THERMALS

**Real-Time Swarm Health:** Operating a fleet of 100+ robots requires streaming latency metrics, IMU tilt distributions, motor temperatures, and fall-rate anomalies to a cloud dashboard.

**The 12-Year-Old Analogy:** A fitness tracker smartwatch for your robot. If motor #4 gets too warm, the dashboard alerts the technician before the plastic melts.

- **Predictive Diagnostics:** Detects mechanical gear friction and battery degradation weeks before failure.
- **Swarm Fleet Sync:** Simultaneously updates entire fleets with synchronized policy rollout waves.
- **Zero-Trust Mesh:** mTLS encrypted MQTT/gRPC telemetry ensures fleet communication cannot be snooped.

### ■ CONGRATULATIONS! COMPLETE PHYSICAL AI MASTERCLASS GRADUATION

- **Full Stack Mastery:** Hardware (P1)  $\rightarrow$  MuJoCo (P2)  $\rightarrow$  PPO RL (P3)  $\rightarrow$  ONNX Clamping (P4)  $\rightarrow$  Rust Daemons (P5)  $\rightarrow$  DevSecOps (P6).
- **Production Ready:** You have architected an enterprise-grade, secure, 50Hz real-time robotics deployment pipeline.
- **Next Step:** Connect to physical hardware via robotctl and watch the Microduck walk!

# ■ PHASE 6 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of DevSecOps, OTA Updates, Cryptography, and Fleet Observability.

5 Questions

## ■ Q1: Atomic A/B Partition Updates

MULTIPLE CHOICE

What is the purpose of an A/B atomic partition update in updateder?

- A. To double the robot's walking speed.
- B. To safely test a new policy in Slot B with instant auto-rollback to Slot A if the robot trips.
- C. To install video games on the Rockchip CPU.

## ■ Q2: Automated Physics CI Gates

MULTIPLE CHOICE

Why must new neural policies pass 1,000 headless MuJoCo simulations in CI?

- A. To mathematically guarantee the model does not trip or spike motor torques before touching real hardware.
- B. Because physical robots do not have batteries.
- C. To make the simulation render faster in 3D.

## ■ Q3: Cryptographic Model Signatures

MULTIPLE CHOICE

How does cryptographic ED25519 signing protect an edge robot?

- A. It prevents the robot from getting wet in the rain.
- B. It ensures the robot only executes verified neural weights signed by authorized DevOps engineers, blocking rogue models.
- C. It encrypts the camera lens.

## ■ Q4: Fleet Observability & Metrics

MULTIPLE CHOICE

Why is real-time telemetry observability critical for a fleet of robots?

- A. It tracks motor temperatures, latency jitter, and fall rates to detect mechanical wear before catastrophic failure.
- B. To broadcast the robot's location to social media.
- C. It reduces the physical weight of the robot.

## ■ Q5: DevSecOps Release Pipeline

MULTIPLE CHOICE

In the 5-stage DevSecOps lifecycle, what happens after automated CI testing passes?

- A. The model is immediately deleted.
- B. The model is cryptographically signed and packaged for secure OTA deployment.
- C. The robot shuts down permanently.

**Answer Key:** 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Safe A/B rollback; 2-A: Automated physics safety; 3-B: Cryptographic trust; 4-A: Predictive diagnostics; 5-B: Secure OTA release)